

Q1: What are Python decorators and where have you used them?

[P-380, O-269]

Answer:

Python decorators are functions that take another function or method and extend or modify its behavior without changing its source code. They're often used for cross-cutting concerns like logging, authentication, caching, and validation. I've used them extensively to keep such concerns reusable and separate from core business logic.

Key Points:

- 1) A decorator wraps a function (or method/class), typically using the `@decorator_name` syntax, to add behavior before/after the original call.
- 2) Common real-world uses: logging, timing, retry logic, access control, caching, and input/output validation.
- 3) They promote DRY principles and clean separation of concerns by centralizing repeated logic in one place.

Trade-offs (if relevant):

Decorators can obscure control flow and make debugging harder if overused or poorly named; they also add indirection that new team members must understand.

Explanation:

For example, I've written a `@require_auth` decorator on API endpoints to verify tokens and handle unauthorized responses, and a `@timeit` decorator to measure latency of critical functions in a service, without touching the core business logic of those functions.

Q2: Explain Python generators and their benefits.

[P-377, O-298]

Answer:

Python generators are functions that use ``yield`` to produce a sequence of values lazily, returning an iterator instead of computing everything at once. They allow you to iterate over potentially large or infinite sequences without storing them all in memory.

Key Points:

- 1) Generators use ``yield`` to produce values one at a time, resuming state between calls instead of restarting the function.
- 2) They are memory-efficient because they generate items on demand rather than building full lists in memory.
- 3) They enable cleaner, more readable code for pipelines (e.g., streaming data, file processing) by composing multiple generator steps.
- 4) They support infinite or very large sequences that would be impossible to materialize as lists.
- 5) Generator expressions (e.g., ``(x*x for x in data)``) provide a concise syntax similar to list comprehensions but lazy.

Trade-offs (if relevant):

Harder to debug than simple loops or lists, can only be iterated once, and you lose random access since values are produced sequentially.

Explanation:

For example, when processing a 10 GB log file, a generator that reads and parses one line at a time lets you stream and filter records without loading the entire file into memory, keeping resource usage low and code simple.

Q3: What is the difference between multiprocessing and multithreading?

[P-377, O-313]

Answer:

Multiprocessing uses multiple processes, each with its own memory space, while multithreading uses multiple threads within a single process sharing the same memory. Multiprocessing avoids the GIL in languages like Python and is better for CPU-bound tasks, whereas multithreading is typically better for I/O-bound tasks.

Key Points:

- 1) Multiprocessing: separate processes, separate memory, true parallelism for CPU-bound work, higher overhead for process creation and inter-process communication.
- 2) Multithreading: multiple threads in one process, shared memory, cheaper context switches, but subject to the GIL and data races if not synchronized.
- 3) Multiprocessing is more robust to crashes (one process can die without killing others), while multithreading risks bringing down the entire process on a fatal error.

Trade-offs (if relevant):

Multiprocessing gains parallel CPU performance and isolation at the cost of higher memory use and communication overhead; multithreading is lighter and simpler to share data but is more error-prone (races, deadlocks) and limited by the GIL in some runtimes.

Explanation:

In a Python web scraper, you might use multithreading to handle many simultaneous HTTP requests (I/O-bound), but use multiprocessing to run heavy image-processing or ML inference tasks in parallel across CPU cores without being limited by the GIL.

Q4: What are Python context managers?

[P-375, O-218]

Answer:

Python context managers are constructs that set up a runtime context, run a block of code, and then reliably clean up resources afterward. They are most commonly used with the `with` statement to manage resources like files, locks, or database connections.

Key Points:

- 1) Implemented via `__enter__` and `__exit__` methods in a class or using the `contextlib` module.
- 2) Ensure deterministic setup and teardown, even if exceptions occur inside the `with` block.
- 3) Common use cases include file handling, threading locks, database transactions, and temporary configuration changes.

Trade-offs (if relevant):

N/A

Explanation:

For example, `with open('data.txt') as f:` uses a context manager to open the file, let you read/write safely, and automatically close the file when the block finishes or an error occurs, avoiding resource leaks.

Q5: How does Python memory management work?

[P-376, O-333]

Answer:

Python uses a private heap for all objects and manages it via automatic memory management, primarily reference counting with a cyclic garbage collector. The interpreter abstracts allocation/deallocation away from the user, though understanding it helps optimize performance and avoid memory leaks.

Key Points:

- 1) All Python objects live in a managed heap controlled by the Python memory manager, not directly by user code.
- 2) Reference counting frees objects immediately when their reference count drops to zero.
- 3) A cyclic garbage collector periodically detects and cleans up reference cycles that reference counting alone cannot handle.
- 4) Memory allocators (like pymalloc) optimize small object allocation and reuse to reduce overhead.
- 5) Large objects may be returned to the OS less eagerly, leading to apparent RSS not shrinking immediately.
- 6) Developers can influence memory usage with object reuse, weak references, and tools like ``gc`` and ``tracemalloc``.

Trade-offs (if relevant):

Automatic management simplifies development but adds overhead; reference counting gives predictable deallocation but needs an additional GC pass to handle cycles, and fragmentation/allocator behavior can make process memory less transparent.

Explanation:

In a long-running web service written in Python, objects for requests and responses are constantly created and destroyed. Python's reference counting frees most objects as soon as each request completes, while the cyclic GC periodically cleans up any accidentally created reference cycles between complex objects like ORM entities and caches, helping control memory growth without manual ``free`` calls.

Q6: Explain the GIL and how it impacts performance.

[P-378, O-271]

Answer:

The Global Interpreter Lock (GIL) is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It mainly affects CPU-bound multithreaded programs, limiting true parallelism on multi-core CPUs, but has less impact on I/O-bound workloads.

Key Points:

- 1) GIL ensures only one thread runs Python bytecode at once, simplifying memory management and avoiding many race conditions.
- 2) For CPU-bound tasks, Python threads do not scale across cores; you typically need multiprocessing or native extensions (C, NumPy) for parallel speedups.
- 3) For I/O-bound tasks (network, disk), threads can still provide good concurrency because the GIL is released during blocking I/O.

Trade-offs (if relevant):

Simpler interpreter implementation and safer memory management vs. poor CPU-bound multithreading performance and limited multi-core utilization.

Explanation:

In a web scraper doing heavy HTML parsing in pure Python, adding more threads won't speed up the CPU-heavy parsing because of the GIL; you'd switch to multiprocessing or move parsing to a C extension to leverage all cores effectively.

Q7: What are Python dataclasses?

[P-375, O-253]

Answer:

Python dataclasses are a feature (introduced in Python 3.7) that automatically generate boilerplate methods like `__init__`, `__repr__`, and `__eq__` for classes that primarily store data. They make it easier and cleaner to define data containers with type hints and default values.

Key Points:

- 1) Declared with the `@dataclass` decorator to auto-generate methods (`init`, `repr`, `eq`, etc.).
- 2) Work best with type-annotated fields and support defaults, `default_factory`, and field metadata.
- 3) Provide options like `frozen=True` (immutability) and `order=True` (comparison methods), improving clarity over plain dictionaries or ad-hoc classes.

Trade-offs (if relevant):

Less flexible than fully custom classes if you need highly customized initialization logic; small runtime overhead versus bare classes, but usually negligible.

Explanation:

For example, when modeling a User entity in a web app, using a dataclass lets you define attributes (`id`, `name`, `email`) succinctly while automatically getting a constructor and readable string representation for logging and debugging.

Q8: What is the difference between a list, tuple, and set in Python?

[P-376, O-339]

Answer:

Lists, tuples, and sets are all Python collections but differ in mutability, ordering, and uniqueness. Lists are mutable and ordered, tuples are immutable and ordered, and sets are mutable, unordered, and store unique elements.

Key Points:

- 1) List: ordered, mutable, allows duplicates; syntax: [1, 2, 2].
- 2) Tuple: ordered, immutable, allows duplicates; syntax: (1, 2, 2); good for fixed data or dictionary keys.
- 3) Set: unordered, mutable, no duplicates; syntax: {1, 2}; supports fast membership tests and set operations.
- 4) Performance: sets give $O(1)$ average membership tests; lists are $O(n)$; tuples can be slightly faster to iterate than lists.
- 5) Use lists for dynamic sequences, tuples for fixed structured records, sets for uniqueness and membership checks.

Trade-offs (if relevant):

Lists are flexible but slower for membership checks than sets; tuples give immutability and safety but can't be changed after creation; sets lose element order and don't allow duplicates, which can be a limitation if order matters.

Explanation:

When processing user IDs, you might first store them in a list as they arrive, then convert to a set to remove duplicates and quickly check membership, and finally store user profiles as tuples (id, name, email) when you need fixed, structured, read-only records.

Q9: Explain deep copy and shallow copy.

[P-376, O-278]

Answer:

Shallow copy creates a new object but reuses references to the same nested objects, while deep copy creates a completely independent clone of the object and all its nested objects. Shallow copies are faster and use less memory but can cause side effects if nested data is modified; deep copies avoid this but are more expensive.

Key Points:

- 1) Shallow copy: new container, same references for inner objects (e.g., `list.copy()`, `copy.copy` in Python).
- 2) Deep copy: recursively copies all nested objects, making a fully independent structure (e.g., `copy.deepcopy` in Python).
- 3) Shallow copies risk unintended shared mutations; deep copies cost more CPU and memory but are safer for isolation.

Trade-offs (if relevant):

Shallow copy: better performance, higher risk of shared-state bugs. Deep copy: safer isolation, higher time and memory cost, and possible issues with complex or non-copyable objects.

Explanation:

In a web service, if you shallow-copy a default configuration dict and then mutate a nested list for one request, it might affect all requests sharing that list. Using a deep copy ensures each request gets its own independent configuration, preventing cross-request interference.

Q10: What is monkey patching in Python?

[P-377, O-232]

Answer:

Monkey patching in Python is the practice of dynamically modifying or extending code at runtime, typically by reassigning attributes or methods of existing classes, modules, or instances. It's often used for quick fixes, testing, or instrumentation, but can make code harder to understand and maintain.

Key Points:

- 1) Done by reassigning functions, methods, or attributes on existing objects or modules at runtime.
- 2) Commonly used in tests to mock dependencies or to hot-fix behavior without changing the original source.
- 3) Can lead to fragile, non-obvious behavior and should be used sparingly and clearly documented.

Trade-offs (if relevant):

Useful for testing and rapid fixes but harms readability, makes debugging harder, and can break with library updates.

Explanation:

For example, in tests you might monkey patch `requests.get` with a fake function that returns a controlled response, allowing you to test your code's logic without making real network calls.